NLP 243 Deep Learning for Natural Language Processing

Fall 2023

Contextual Embeddings

Outline for Today

- Contextual Embeddings
- ELMo - Embeddings from Language Models
- Encoder-Decoder Architectures
- Sequence-to-Sequence Models
- Beam Search

Outline for Today

- Contextual Embeddings
- ELMo - Embeddings from Language Models
- Encoder-Decoder Architectures
- Sequence-to-Sequence Models
- Beam Search

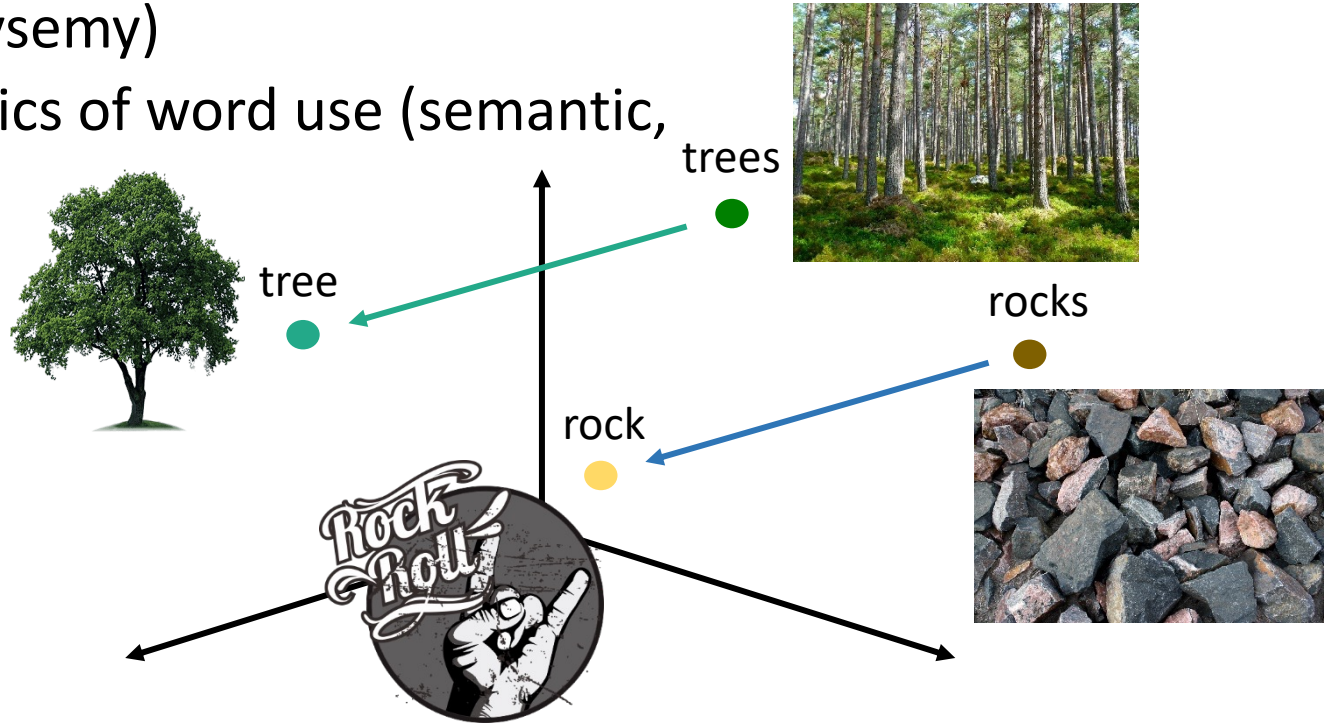
Contextual Embeddings

Pre-train embeddings using **large unlabeled** datasets. Two strategies for applying pre-trained language representations to downstream tasks:

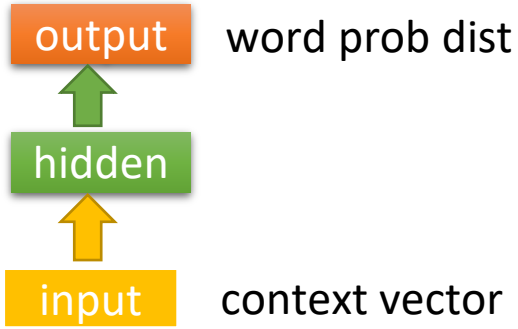
- **Feature-based approach**, uses task-specific architectures that include the pre-trained representations as additional features (i.e., ELMo (Peters et al., 2018)).
- **Fine-tuning approach**, introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning all pretrained parameters, using **smaller** datasets with task-specific labels (i.e., the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018) or BERT (Devlin et al, 2019)).

Word Representations

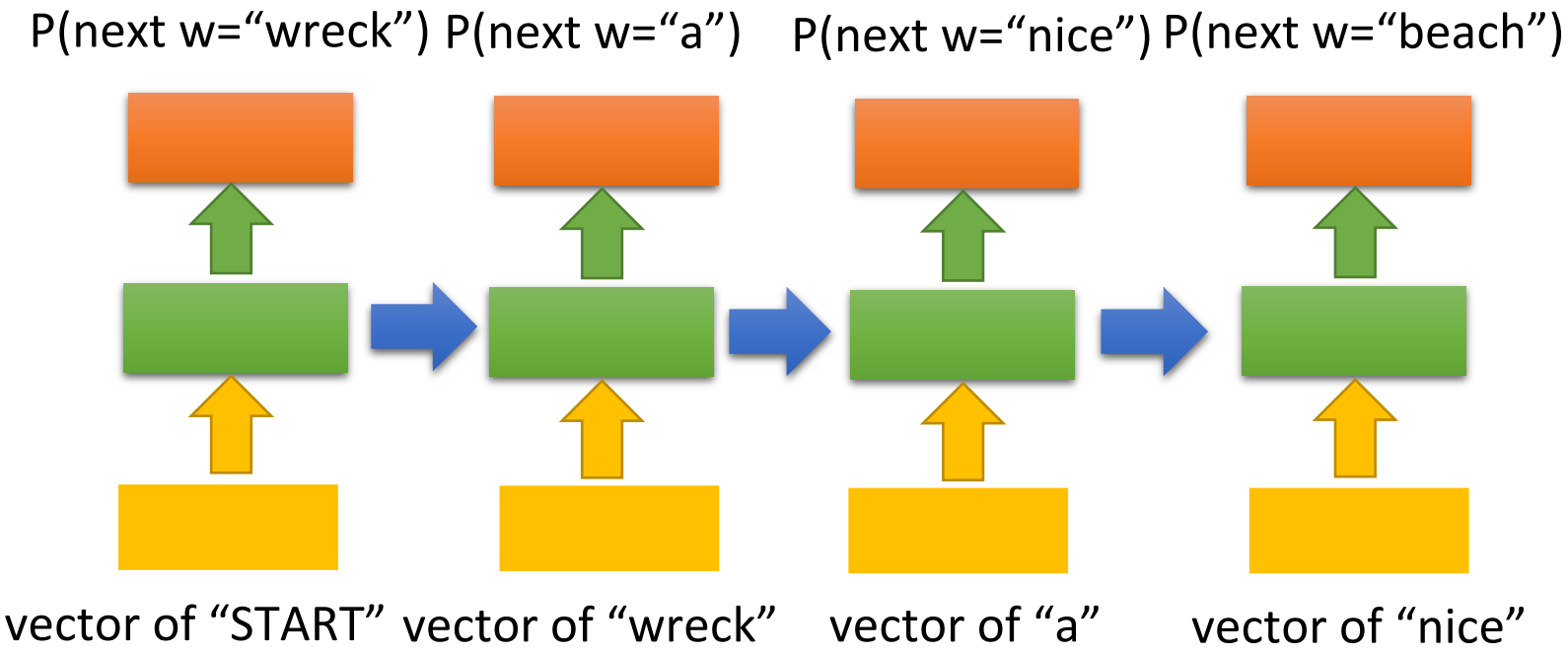
- Assumption: one representation for each word
- Issues:
  - Multiple **senses** (polysemy)
  - Complex characteristics of word use (semantic, syntactic, etc.)



RNN-LM



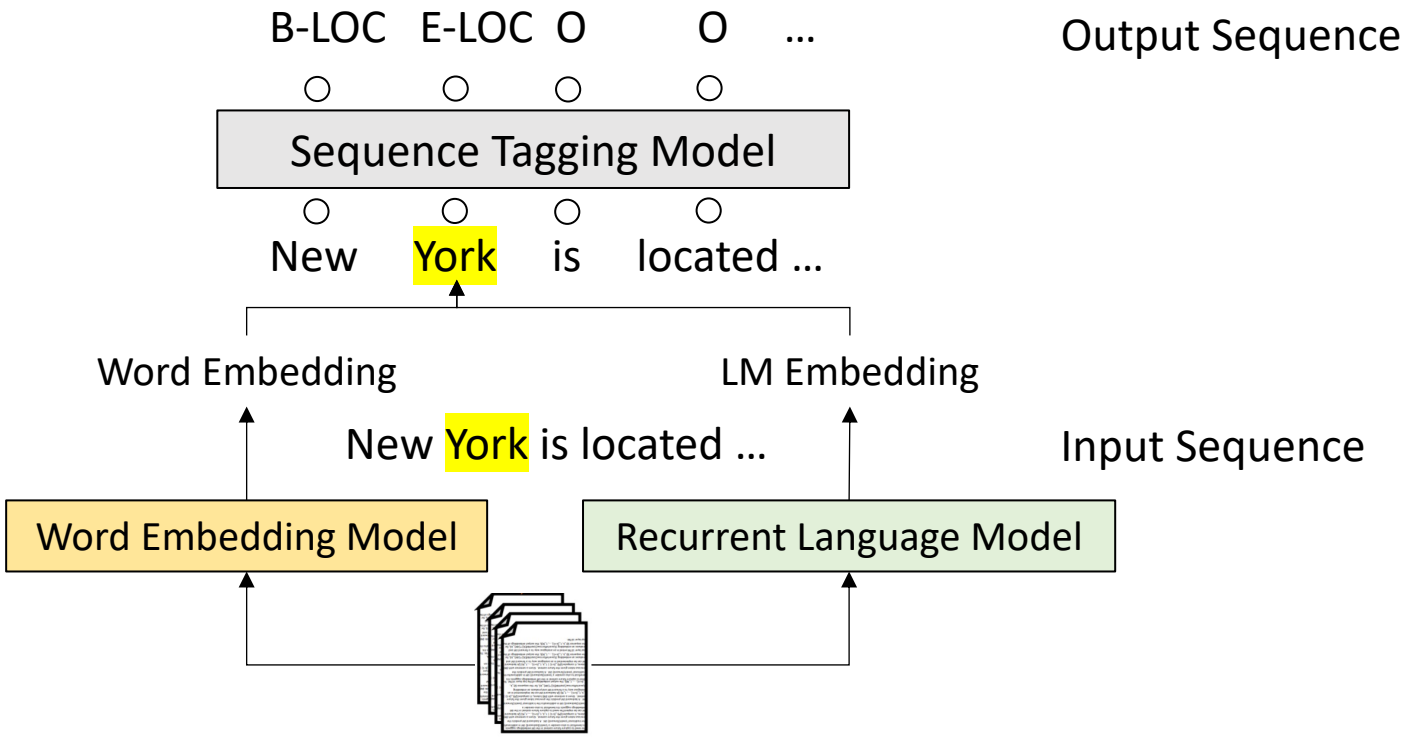
- Idea: condition the neural network on <u>all previous words</u> and <u>tie the weights</u> at each time step



The LM produces **context-specific word representations** at each position

TagLM

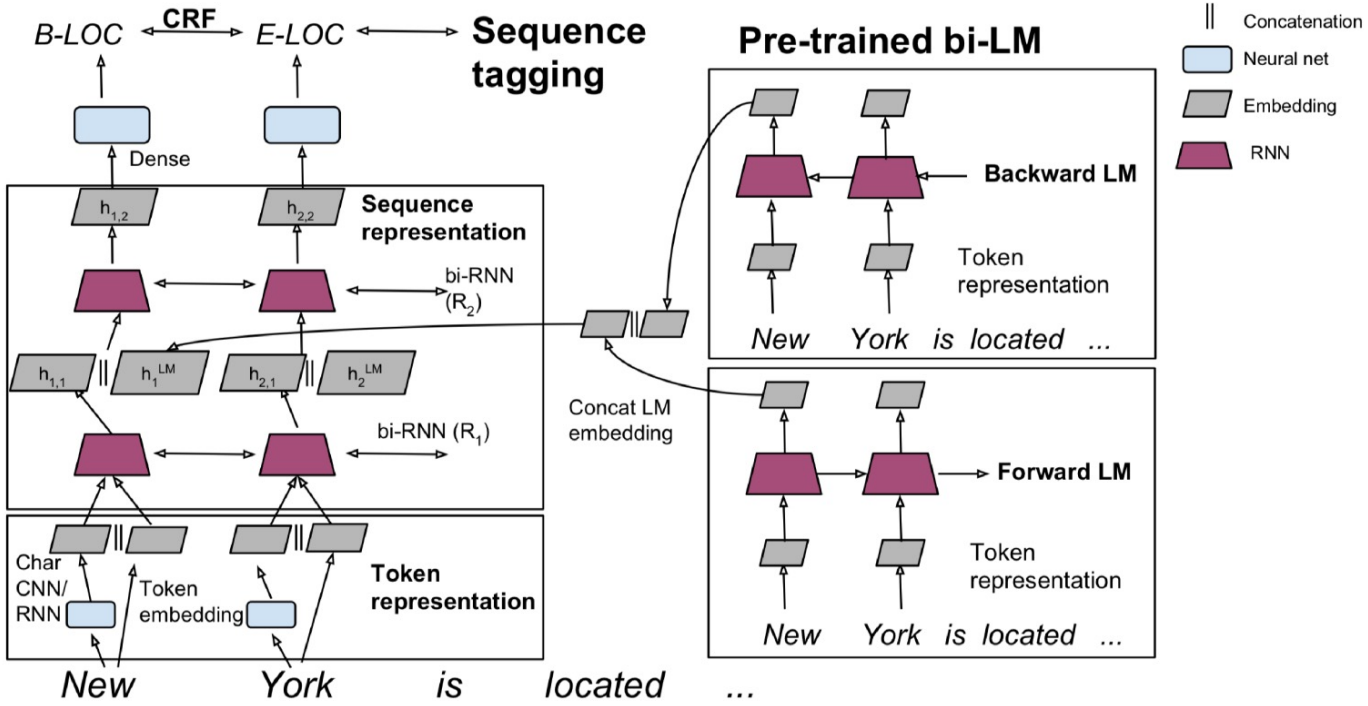
- **Idea:** train (contextual) NLM on big unannotated data and provide the <u>context-specific embeddings</u> for the target task → **semi-supervised learning**
- Named entity recognition (NER) and chunking



Peters et al., "Semi-supervised sequence tagging with bidirectional language models," in *ACL*, 2017.

TagLM Model Details

- Leveraging pre-trained LM information



$$h_{k,1} = [\overrightarrow{h}_{k,1}; \overleftarrow{h}_{k,1}; h_k^{LM}]$$

TagLM Steps

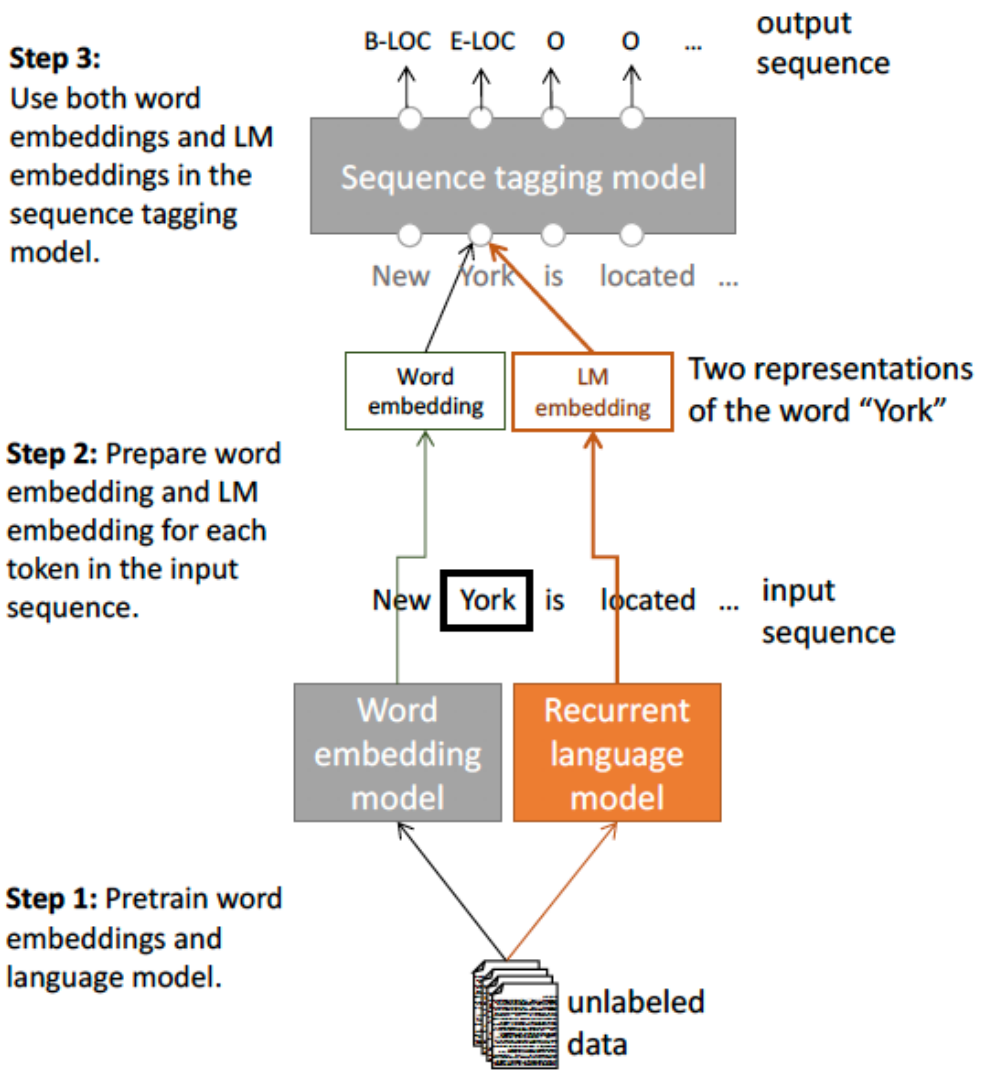
- Step 1 can use very large, unlabeled dataset.
- Steps 2 and 3 use the training data labeled for the task.

- CoNLL 2003 NER English data (Sang and De Meulder, 2003)

English data	Articles	Sentences	Tokens
Training set	946	14,987	203,621
Development set	216	3,466	51,362
Test set	231	3,684	46,435

English data	LOC	MISC	ORG	PER
Training set	7140	3438	6321	6600
Development set	1837	922	1341	1842
Test set	1668	702	1661	1617

- 1B Word Benchmark dataset (Chelba et al, 2014)
  - using 800M for LM training.



TagLM for Named Entity Recognition

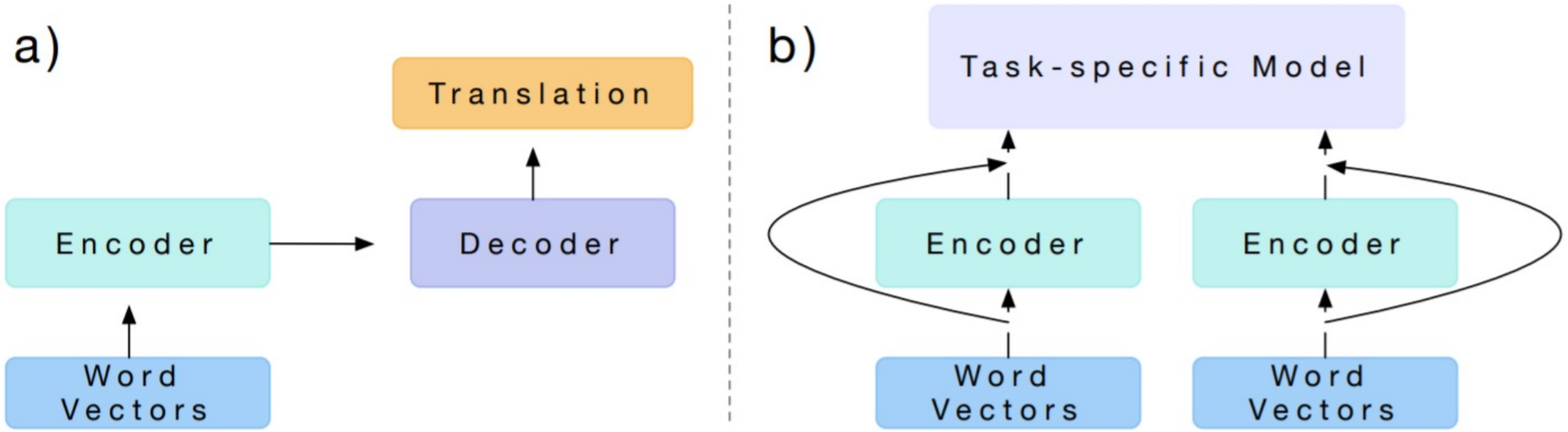
- Find and classify names in text

The decision by the independent MP Andrew Wilkie to withdraw his support for the minority Labor government sounded dramatic but it should not further threaten its stability. When, after the 2010 election, Wilkie, Rob Oakeshott, Tony Windsor and the Greens agreed to support Labor, they gave just two guarantees: confidence and supply.

Model	Description	CONLL 2003 F1
Peters+, 2017	BLSTM + char CNN + CRF	90.87
Ma and Hovy, 2016	BLSTM + char CNN + CRF	91.21
TagLM (Peters+, 2017)	LSTM BiLM in BLSTM Tagger	**91.93**

CoVe - Context Vectors

- **Idea:** use trained sequence model to provide contexts to other NLP models
  a) MT is to capture the meaning of a sequence
  b) NMT provides the context for target tasks



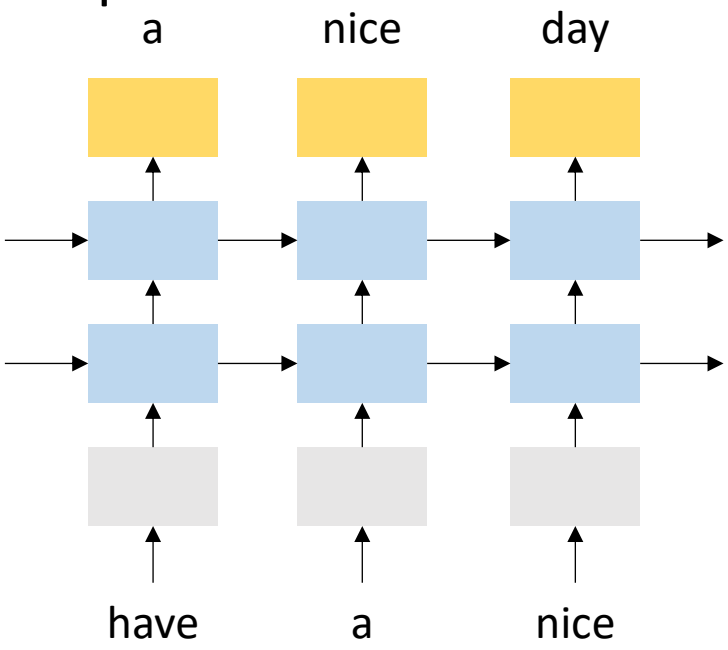
McCann et al., "Learned in Translation: Contextualized Word Vectors", in *NIPS*, 2017.

Outline for Today

- Contextual Embeddings
- ELMo - Embeddings from Language Models
- Encoder-Decoder Architectures
- Sequence-to-Sequence Models
- Beam Search

ELMo: Embeddings from Language Models

- **Idea:** contextualized word representations
  - Learn word vectors using <u>long contexts</u> instead of a context window
  - Learn a deep Bidirectional-LM and use <u>all its layers</u> in prediction



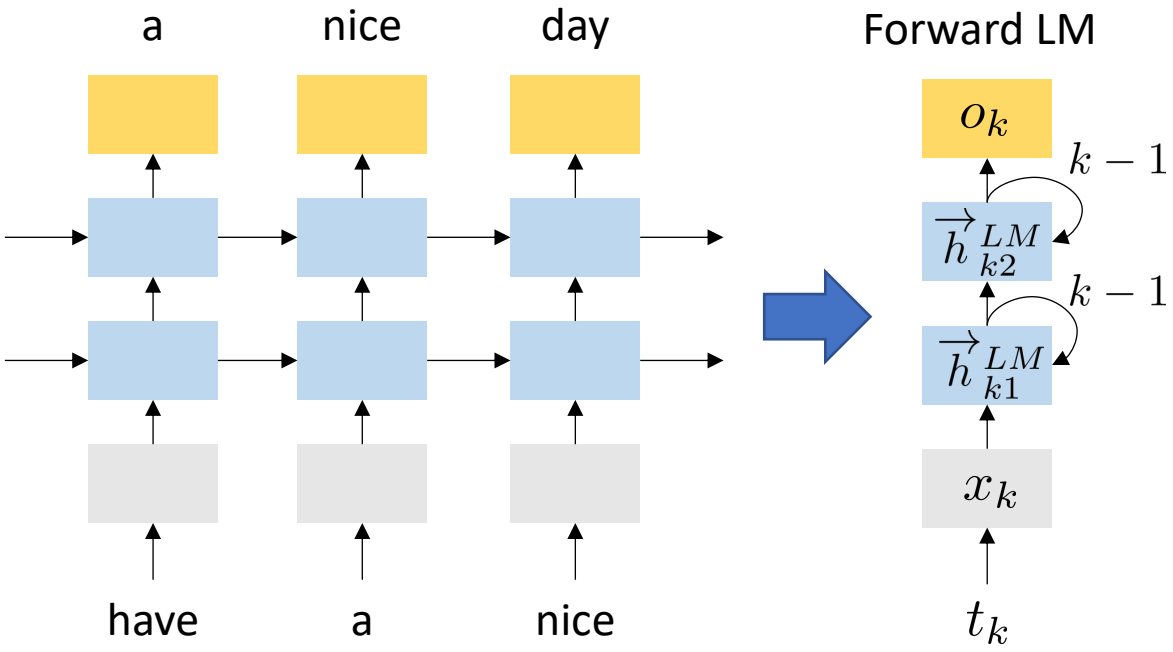
Each token is assigned a representation that is a function of entire input sentence.

Peters et al., "Deep Contextualized Word Representations", in *NAACL-HLT*, 2018.

ELMo: Embeddings from Language Models

1) Bidirectional LM

$$p(t_1, t_2, \cdots, t_N) = \prod_{k=1}^{N} p(t_k \mid t_1, \cdots, t_{k-1})$$



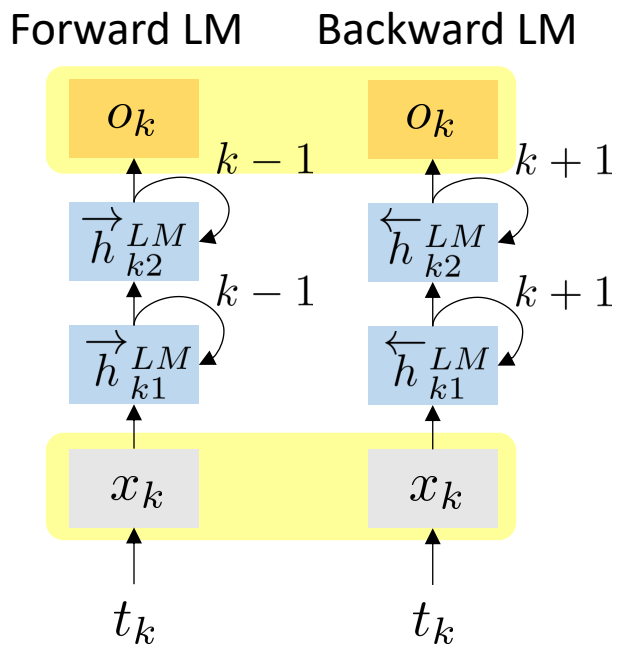
ELMo: Embeddings from Language Models

1) Bidirectional LM

$$p(t_1, t_2, \cdots, t_N) = \prod_{k=1}^{N} p(t_k \mid t_1, \cdots, t_{k-1})$$

$$p(t_1, t_2, \cdots, t_N) = \prod_{k=1}^{N} p(t_k \mid t_{k+1}, \cdots, t_N)$$

- 2 BLSTM layers



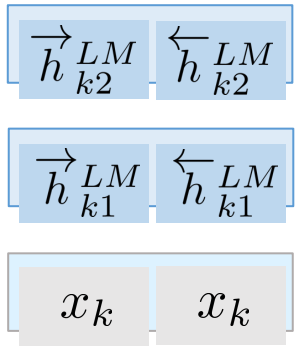
Joint maximization of log likelihood in backward and forward directions.

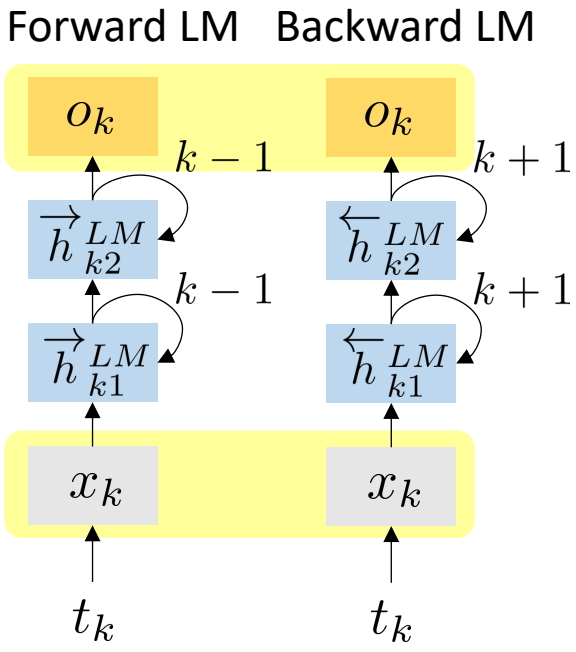
ELMo: Embeddings from Language Models

2) ELMo

- Learn the <u>task-specific linear combination</u> of LM representations
- Use multiple layers in LSTM instead of top one

$$\mathrm{ELMo}_k^{\mathrm{task}} = \gamma^{\mathrm{task}} \times \sum \left\{ \begin{array}{l} s_2^{\mathrm{task}} \times h_{k2}^{LM} \\ s_1^{\mathrm{task}} \times h_{k1}^{LM} \\ s_0^{\mathrm{task}} \times h_{k0}^{LM} \end{array} \right.$$





- γ^{task} scales overall usefulness of ELMo to task
- s^{task} are softmax-normalized weights
- optional layer normalization

Elmo embeddings are deep: A task-specific embedding with combination weights learned from a downstream task.

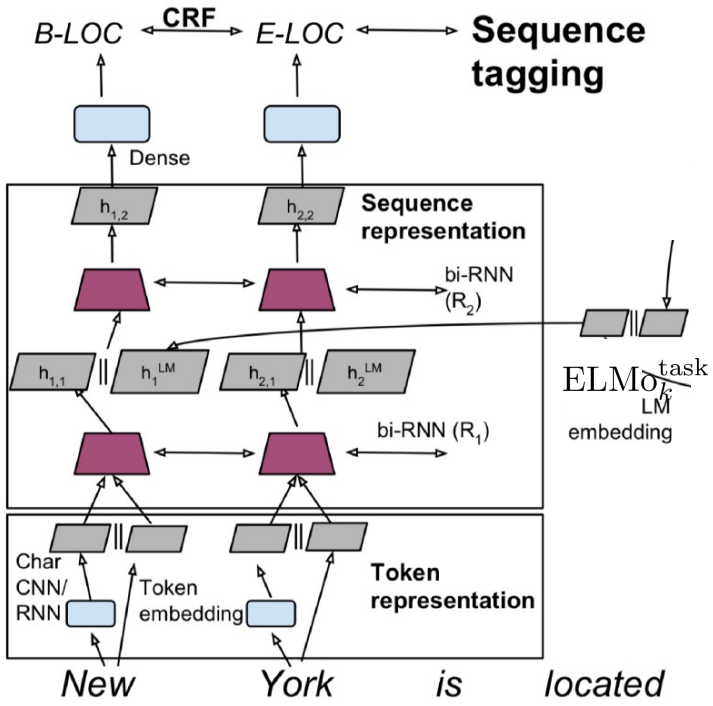
ELMo: Embeddings from Language Models

3) Use ELMo in Supervised NLP Tasks

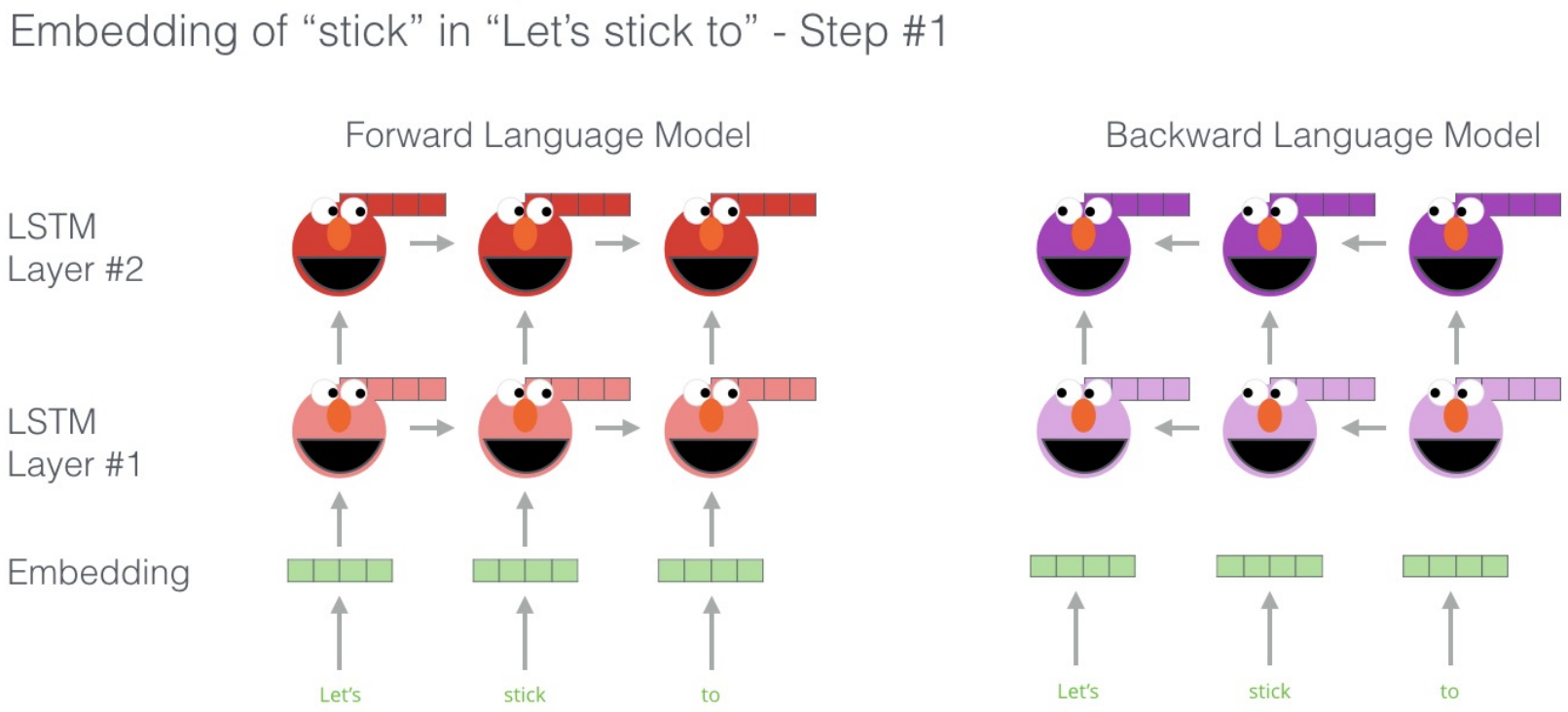
- Get LM embedding for each word
- Freeze the LM weights and form ELMo enhanced embeddings

  $[h_k; \mathrm{ELMo}_k^{\mathrm{task}}]$: concatenate ELMo into the intermediate layer

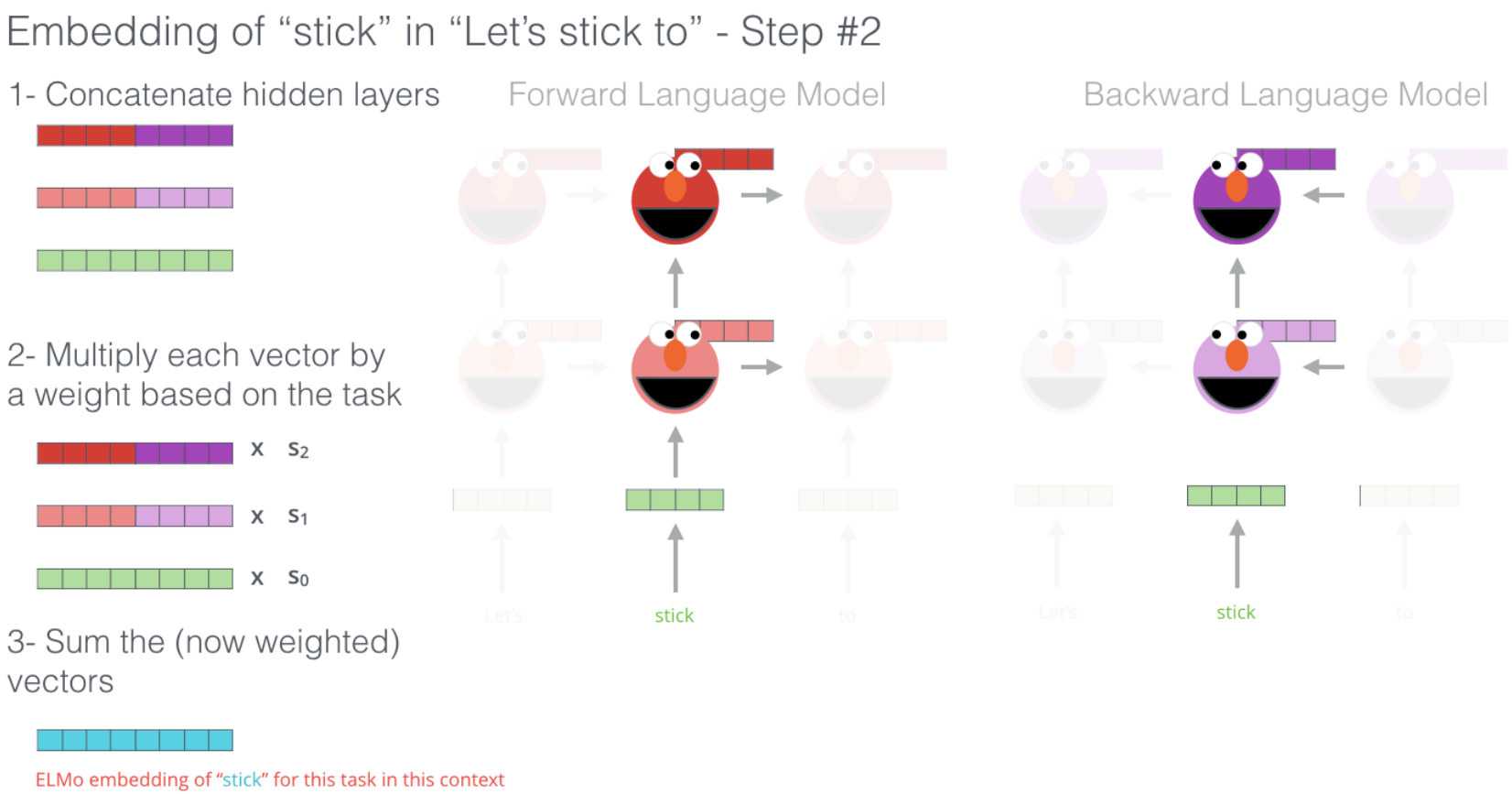
  $[x_k; \mathrm{ELMo}_k^{\mathrm{task}}]$: concatenate ELMo into the input layer



ELMo Illustration



ELMo Illustration



ELMo on Name Entity Recognition

Model	Description	CONLL 2003 F1
Peters+, 2017	BLSTM + char CNN + CRF	90.87
Ma and Hovy, 2016	BLSTM + char CNN + CRF	91.21
TagLM (Peters+, 2017)	LSTM BiLM in BLSTM Tagger	91.93
ELMo (Peters+, 2018)	ELMo in BLSTM	**92.22**

ELMo Results

- Improvement on various NLP tasks

	Task	Previous SOTA		Our baseline	ELMo + baseline	Increase (absolute/ relative)
Machine Reading Comprehension	SQuAD	Liu et al. (2017)	84.4	81.1	85.8	4.7 / 24.9%
Textual Entailment	SNLI	Chen et al. (2017)	88.6	88.0	88.7 ± 0.17	0.7 / 5.8%
Semantic Role Labeling	SRL	He et al. (2017)	81.7	81.4	84.6	3.2 / 17.2%
Coreference Resolution	Coref	Lee et al. (2017)	67.2	67.2	70.4	3.2 / 9.8%
Name Entity Recognition	NER	Peters et al. (2017)	91.93 ± 0.19	90.15	92.22 ± 0.10	2.06 / 21%
Sentiment Analysis	SST-5	McCann et al. (2017)	53.7	51.4	54.7 ± 0.5	3.3 / 6.8%

Good transfer learning in NLP (similar to computer vision)

ELMo Results

	Task	Previous SOTA		Our baseline	ELMo + baseline	Increase (absolute/ relative)
Machine Reading Comprehension	SQuAD	Liu et al. (2017)	84.4	81.1	85.8	4.7 / 24.9%
Textual Entailment	SNLI	Chen et al. (2017)	88.6	88.0	88.7 ± 0.17	0.7 / 5.8%
Semantic Role Labeling	SRL	He et al. (2017)	81.7	81.4	84.6	3.2 / 17.2%
Coreference Resolution	Coref	Lee et al. (2017)	67.2	67.2	70.4	3.2 / 9.8%
Name Entity Recognition	NER	Peters et al. (2017)	91.93 ± 0.19	90.15	92.22 ± 0.10	2.06 / 21%
Sentiment Analysis	SST-5	McCann et al. (2017)	53.7	51.4	54.7 ± 0.5	3.3 / 6.8%

SRL: Semantic Role Labeling

The task of finding the semantic roles of each argument of each predicate in a sentence.

FrameNet / PropBank:

[You]	can't	[blame]	[the program]	[for being unable to identify it]
COGNIZER		TARGET	EVALUEE	REASON

[The San Francisco Examiner]	issued	[a special edition]	[yesterday]
ARG0	TARGET	ARG1	ARGM-TMP

ELMo Analysis

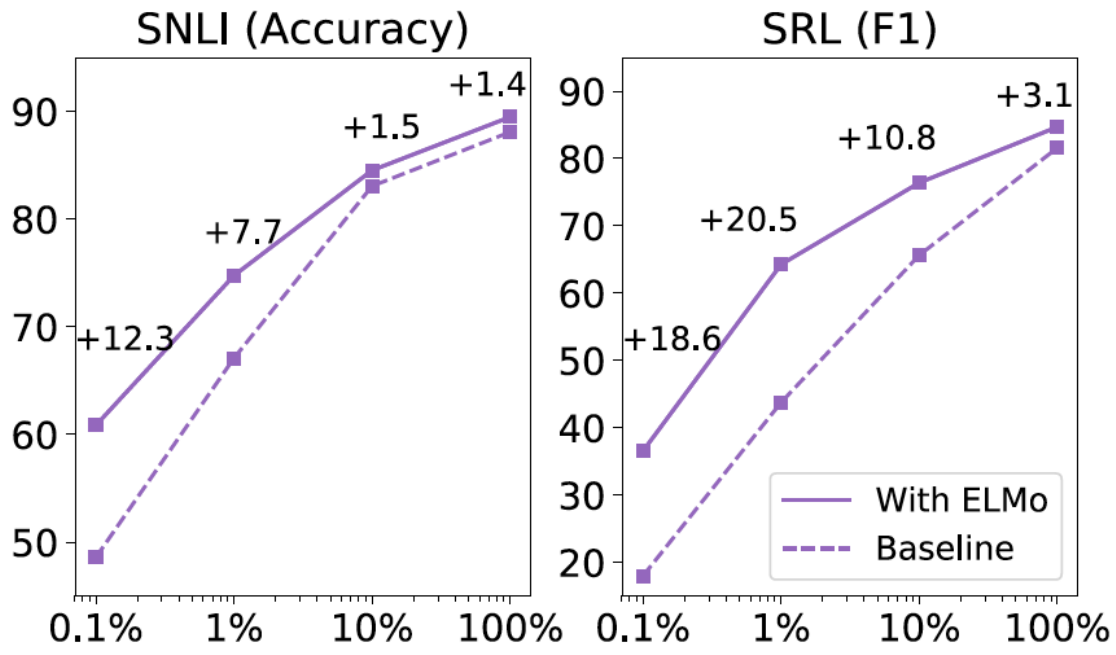
- Word embeddings vs. contextualized embeddings

	Source	Nearest Neighbors
GloVe	play	playing, game, games, played, players, plays, player, Play, football, multiplayer
biLM	Chico Ruiz made a spectacular play on Alusik 's grounder {...}	Kieffer , the only junior in the group , was commended for his ability to hit in the clutch , as well as his all-round excellent play .
	Olivia De Havilland signed to do a Broadway play for Garson {...}	{...} they were actors who had been handed fat roles in a successful play , and had talent enough to fill the roles competently , with nice understatement .

The Bidirectional LM is able to disambiguate both the **PoS** and **word sense** in the source sentence

ELMo Analysis

- Sample Efficiency



ELMo Analysis

- The two NNLM layers have differentiated uses/meanings
  - Lower layer is better for lower-level **syntax**, etc.

  Part-of-speech tagging, syntactic dependencies, NER
  - Higher layer is better for higher-level **semantics**

  Sentiment, Semantic role labeling, question answering, SNLI

PoS Tagging

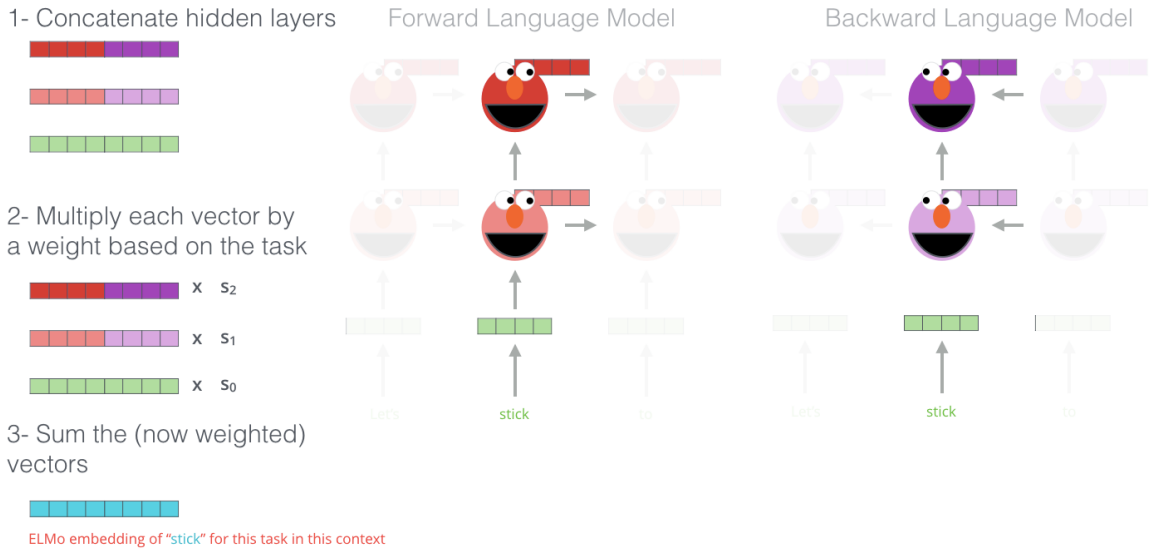
Model	Acc.
Collobert et al. (2011)	97.3
Ma and Hovy (2016)	97.6
Ling et al. (2015)	**97.8**
CoVe, First Layer	*93.3*
CoVe, Second Layer	92.8
biLM, First Layer	*97.3*
biLM, Second Layer	96.8

Word Sense Disambiguation

Model	F_1
WordNet 1st Sense Baseline	65.9
Raganato et al. (2017a)	69.9
Iacobacci et al. (2016)	**70.1**
CoVe, First Layer	59.4
CoVe, Second Layer	*64.7*
biLM, First layer	67.4
biLM, Second layer	*69.0*

ELMo

- Contextualized embeddings learned from LM provide informative cues
- ELMo – a general approach for learning high-quality deep context-dependent representations from bidirectional LMs
  - Pre-trained ELMo: https://allennlp.org/elmo
  - ELMo can process the character-level inputs



Outline for Today

- Contextual Embeddings
- ELMo - Embeddings from Language Models
- Encoder-Decoder Architectures
- Sequence-to-Sequence Models
- Beam Search

11/16 - In Class Poll

Outline for Today

- Contextual Embeddings
- ELMo - Embeddings from Language Models
- Encoder-Decoder Architectures
- Sequence-to-Sequence Models
- Beam Search

Content from an earlier lecture

Variable Length Sequences

- Inputs can have variable length!

- V = {the, a, chased, cat, dog, is, cute}

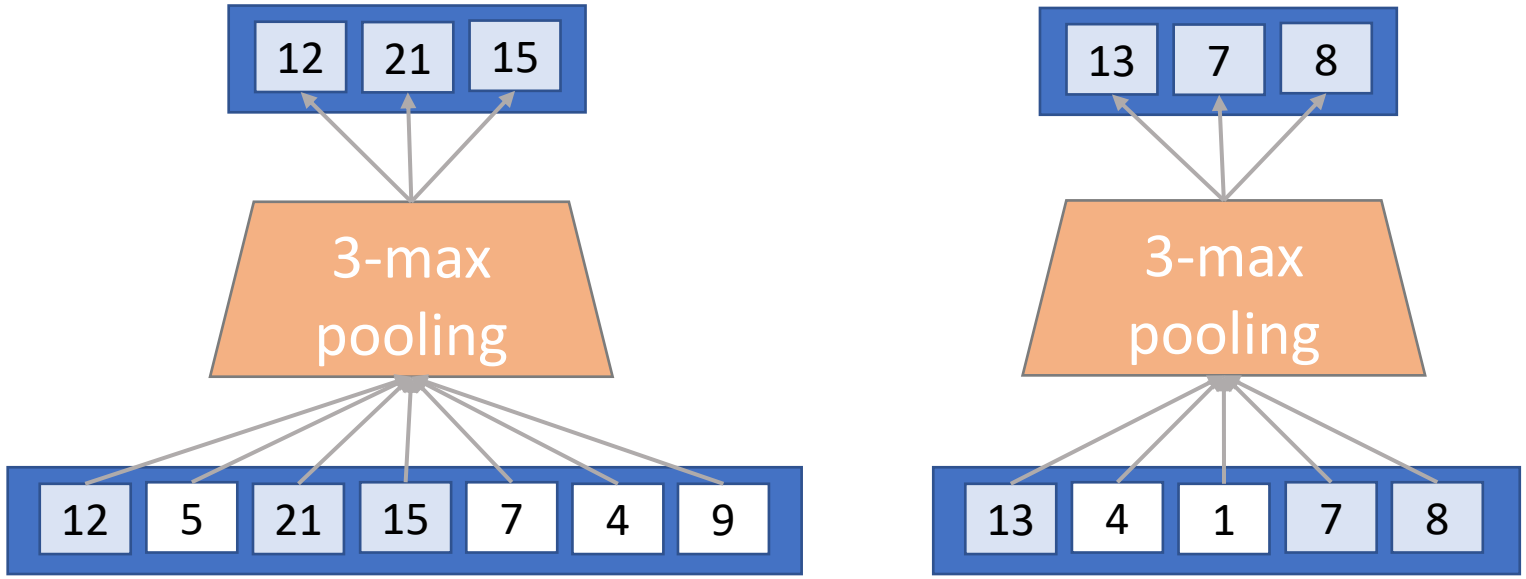
		Bag of words:
• the cat is cute	→	[1 0 0 1 0 1 1]
• the cat chased a dog	→	[1 1 1 1 1 0 0]
• the dog chased a cat	→	[1 1 1 1 1 0 0]

Bag of words approach represents variable length inputs with vectors of size |V|.
Works well for tasks where word ordering is not important.

Content from an earlier lecture

Variable Length Sequences (cont.)

- CNNs with k-max pooling:

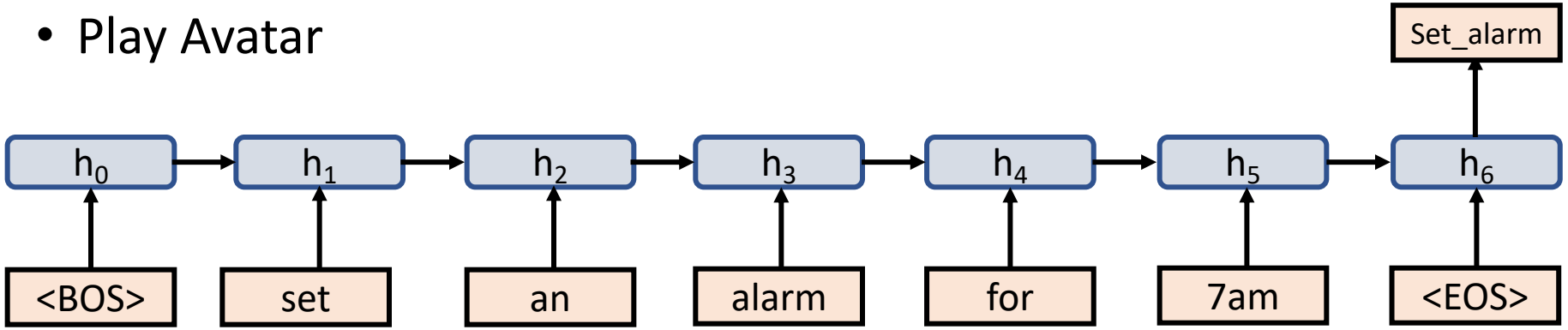


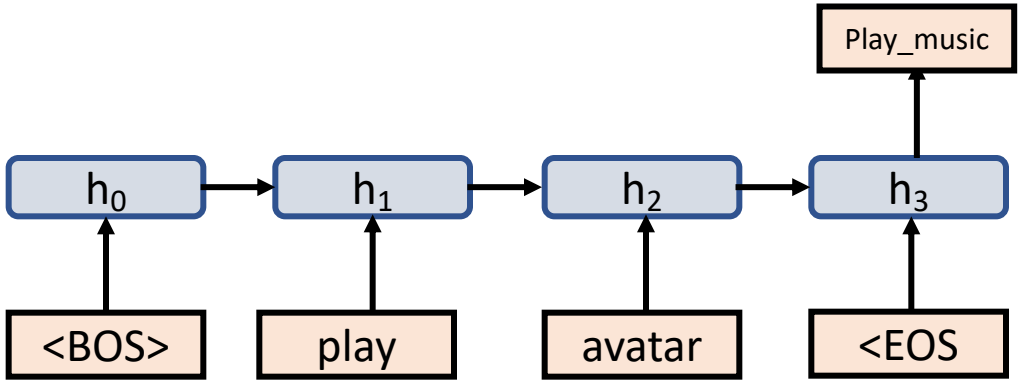
k-max pooling maps all inputs to a fixed number of vectors.

Content from an earlier lecture

Variable Length Sequences (cont.)

- RNNs for intent classification:
  - Set an alarm for 7am
  - Play Avatar





RNNs with padding map all utterances to length k.
Dynamic RNNs allow us unroll to utterance length.

Variable Length Sequences (cont.)

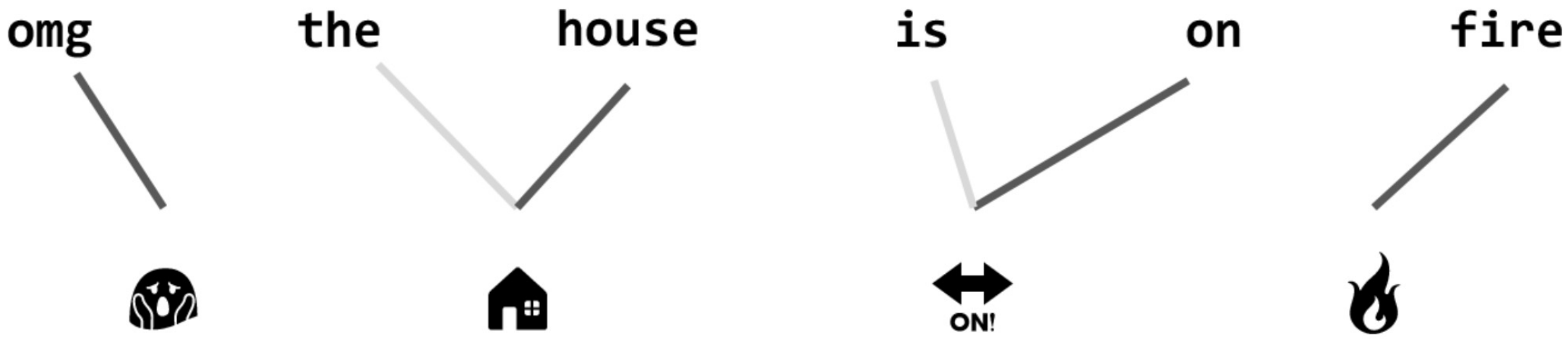
- What if the outputs are a list of tokens of variable length?

- Examples:

Task	Input	Output
nasılsın	Nasılsın? (in Turkish)	How are you?
Summarization	1 or more documents	Summary
Dialogue Systems	Previous conversation A: How are you? B: Thanks, I am doing ok. And you?	Next response A: Thanks, I am fine. Have you seen the Parasite movie? Wanna go see it together tomorrow?

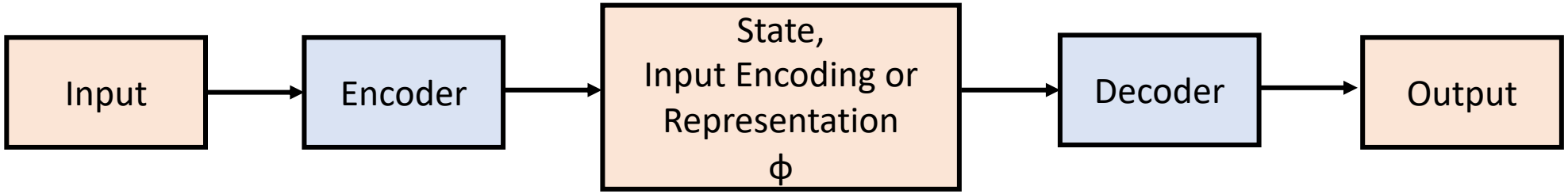
Example: Emoji Translation

- Aiming to converting messages typed on a phone keyboard to emojis.
- Alignment between the input and the output:



Encoder-Decoder Architecture

- Composition of two models:



- Encoder aims to capture important information for the task
- ϕ is usually a vector
- Decoder aims to decode the encoded input to target output
- Trained jointly

Earlier Work: Interlingua-based Machine Translation

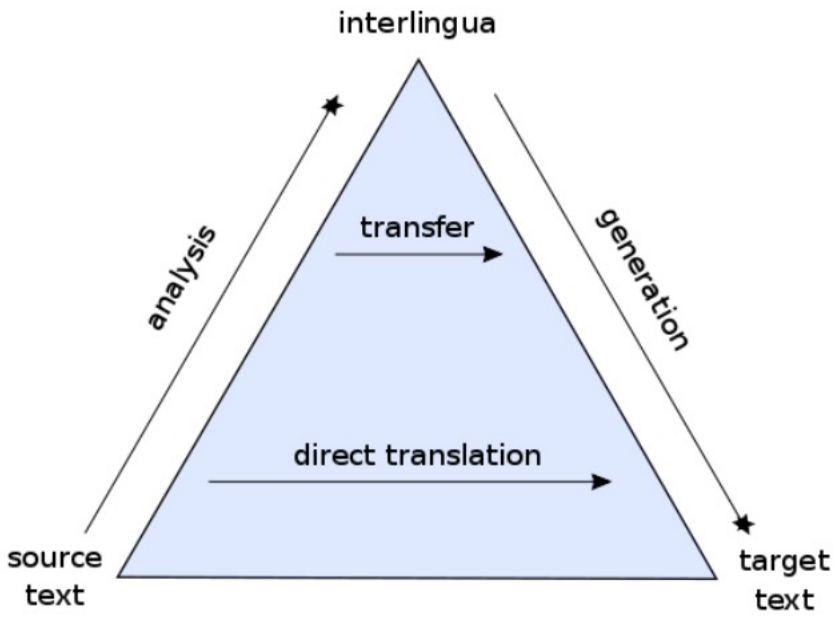


Image from Wikipedia: https://en.wikipedia.org/wiki/Transfer-based_machine_translation

Interlingua:

- Language independent meaning representation
- makes explicit the distinctions necessary for successful translation

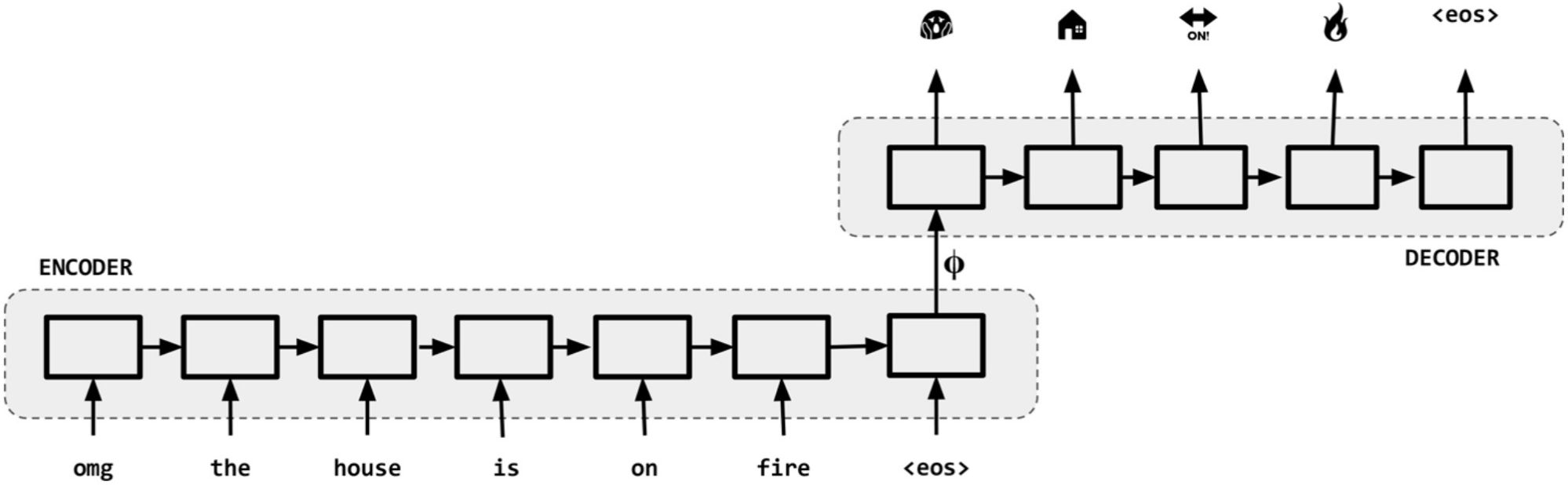
Encoder-decoder models aim to learn representations from data: “Representation learning”

Outline for Today

- Contextual Embeddings
- ELMo - Embeddings from Language Models
- Encoder-Decoder Architectures
- Sequence-to-Sequence Models
- Beam Search

S2S Learning with Neural Networks (cont.)

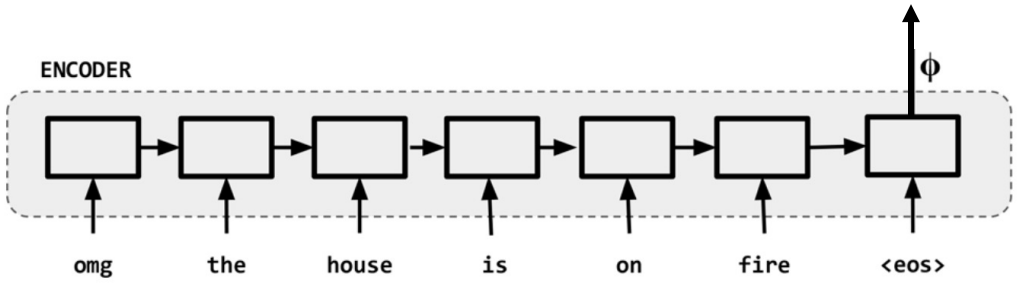
- For NL sequences, the encoder and decoder could both be RNNs.
- (Sutskever et al, 2014)



S2S Learning with Neural Networks: The Encoder

- Input: $x_1,\ldots,x_T$ where x_t is the tth word
- The transformation of the RNN's hidden states, denoted by f: $\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$
- The encoder captures information and encodes it into the context vector ϕ with a function q: $\phi = q(\mathbf{h}_1,\ldots,\mathbf{h}_T)$

$$q(\mathbf{h}_1,\ldots,\mathbf{h}_T) = \mathbf{h}_T$$



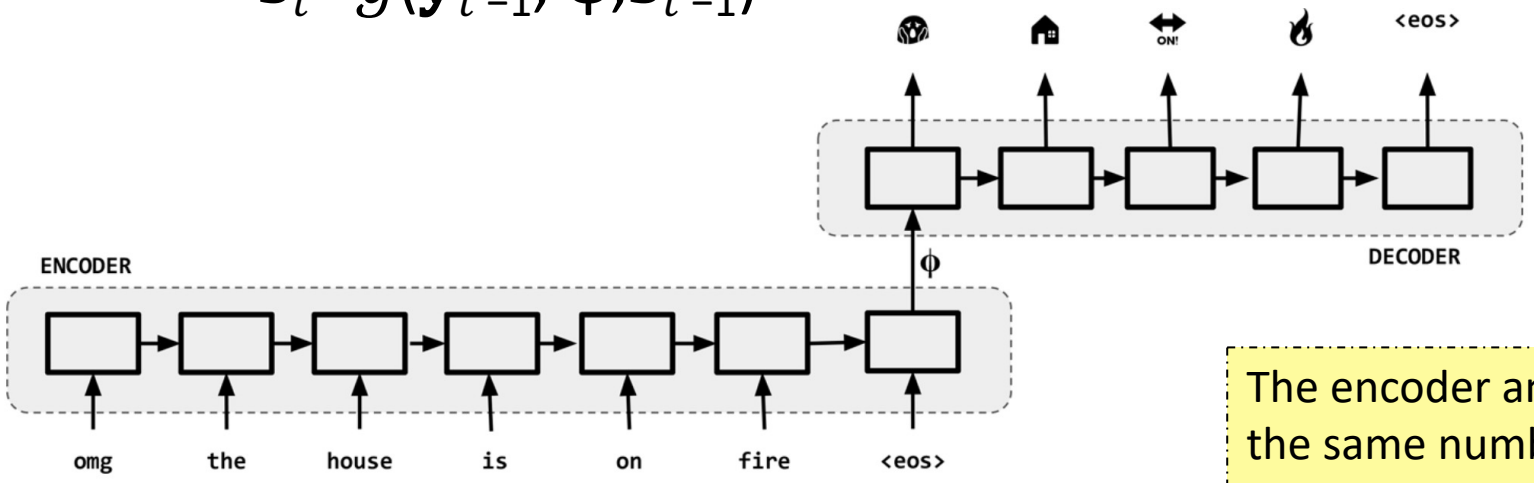
S2S Learning with Neural Networks: The Decoder

- The context vector, ϕ, encodes the information from the complete input sequence $x_1,\ldots,x_T$.
- If given outputs in the training set are $y_1,\ldots,y_{T'}$, at each timestep t',

$$P(y_{t'}|y_1,\ldots,y_{t'-1}, \phi)$$

- Another RNN as decoder, with hidden states s_t

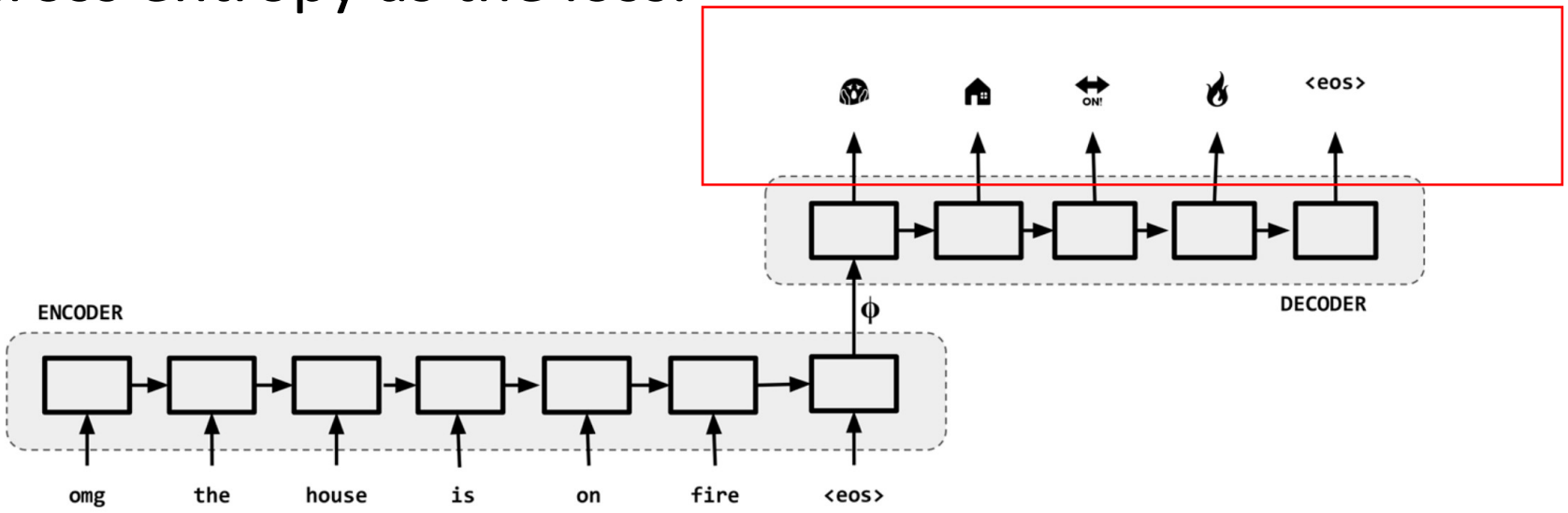
$$\mathbf{s}_{t'}=g(\mathbf{y}_{t'-1}, \phi,\mathbf{s}_{t'-1})$$



The encoder and decoder RNNs have the same numbers of layers and hidden unit size.

S2S Learning with Neural Networks (cont.)

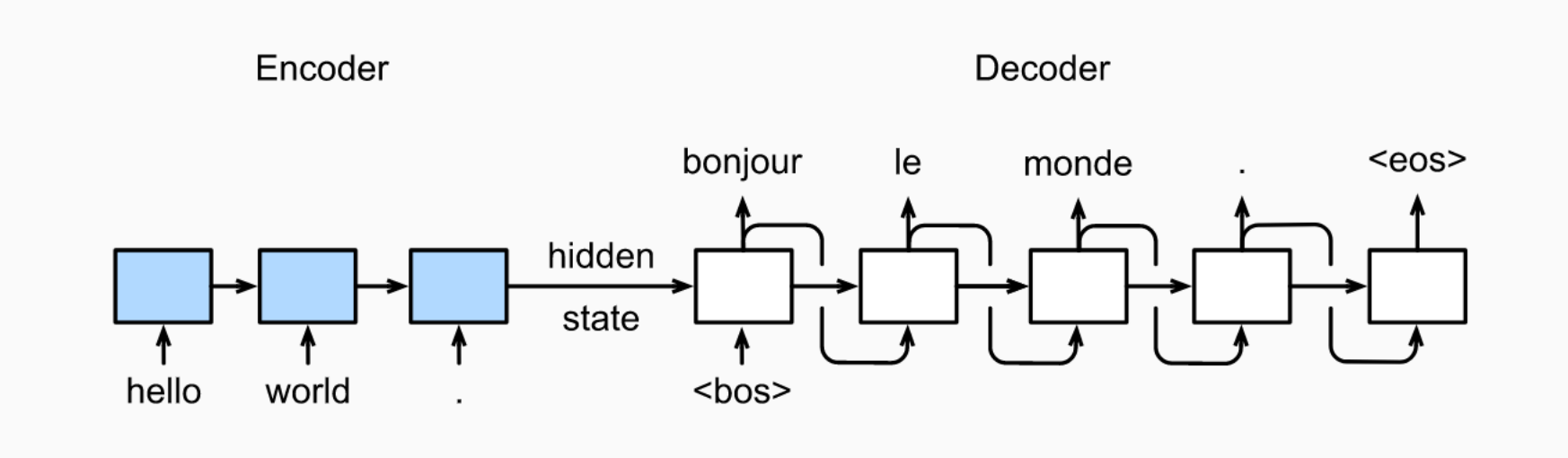
- Decoder has a dense layer after the RNN layers, where the hidden size is the vocabulary size.
- The dense layer predicts the confidence score for each word.
- We can use softmax on the output scores
- And use cross entropy as the loss.



S2S Learning with Neural Networks (cont.)

- Simplest method for predictions, greedy search.

$$y_{t'}=\text{argmax}_{y\in Y}P(y|y_1,\ldots,y_{t'-1},\phi)$$



Generate until the "<eos>" symbol is detected, or the output sequence has reached the max length T.

Issue with Greedy Search

- The conditional probability for generating an output sequence

$$\prod^{T'}_{t'=1} P(y_{t'} | y_1, \ldots, y_{t'-1}, \phi)$$

Timestep	1	2	3	4
A	0.5	0.1	0.2	0.0
B	0.2	0.4	0.2	0.2
C	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

P("ABC<eos>" | …) = 0.048

Timestep	1	2	3	4
A	0.5	0.1	0.1	0.1
B	0.2	0.4	0.6	0.2
C	0.2	0.3	0.2	0.1
<eos>	0.1	0.2	0.1	0.6

P("ACB<eos>" | …) = 0.054

No guarantee that optimal sequence will be obtained with greedy search!

Exhaustive Search

- Examination of all possible sequences.
- Computational overhead: O($|Y|^{T'}$)
- For example, if $|Y|$=10000 and T'=10, we will need to evaluate $10000^{10} = 10^{40}$ sequences

Exhaustive search is too expensive even with small vocabulary sizes!

Outline for Today

- Contextual Embeddings
- ELMo - Embeddings from Language Models
- Encoder-Decoder Architectures
- Sequence-to-Sequence Models
- Beam Search

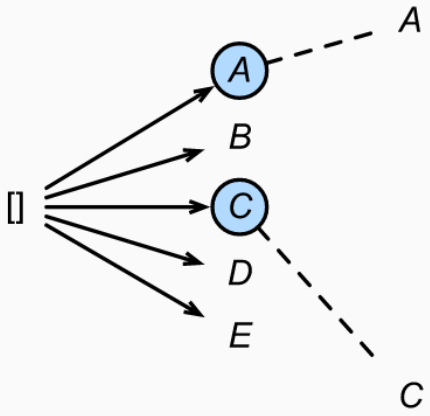
Beam Search

- Beam-size = k
- At time step 1, select top k candidates
- For each subsequent timestep:
  - Determine the $k|Y|$ possible output sequences based on the k candidate output sequences from the previous timestep
  - select the top k output sequences with the highest conditional probability amongst those.

Beam Search (cont.)

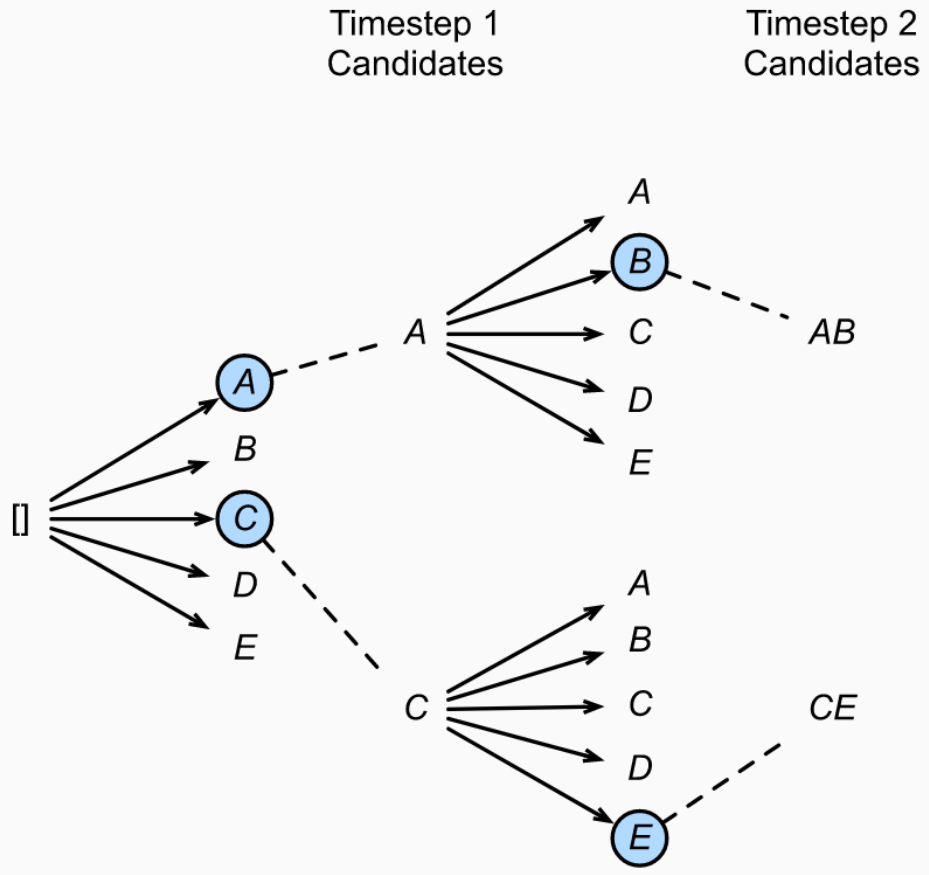
k=2

Timestep 1
Candidates



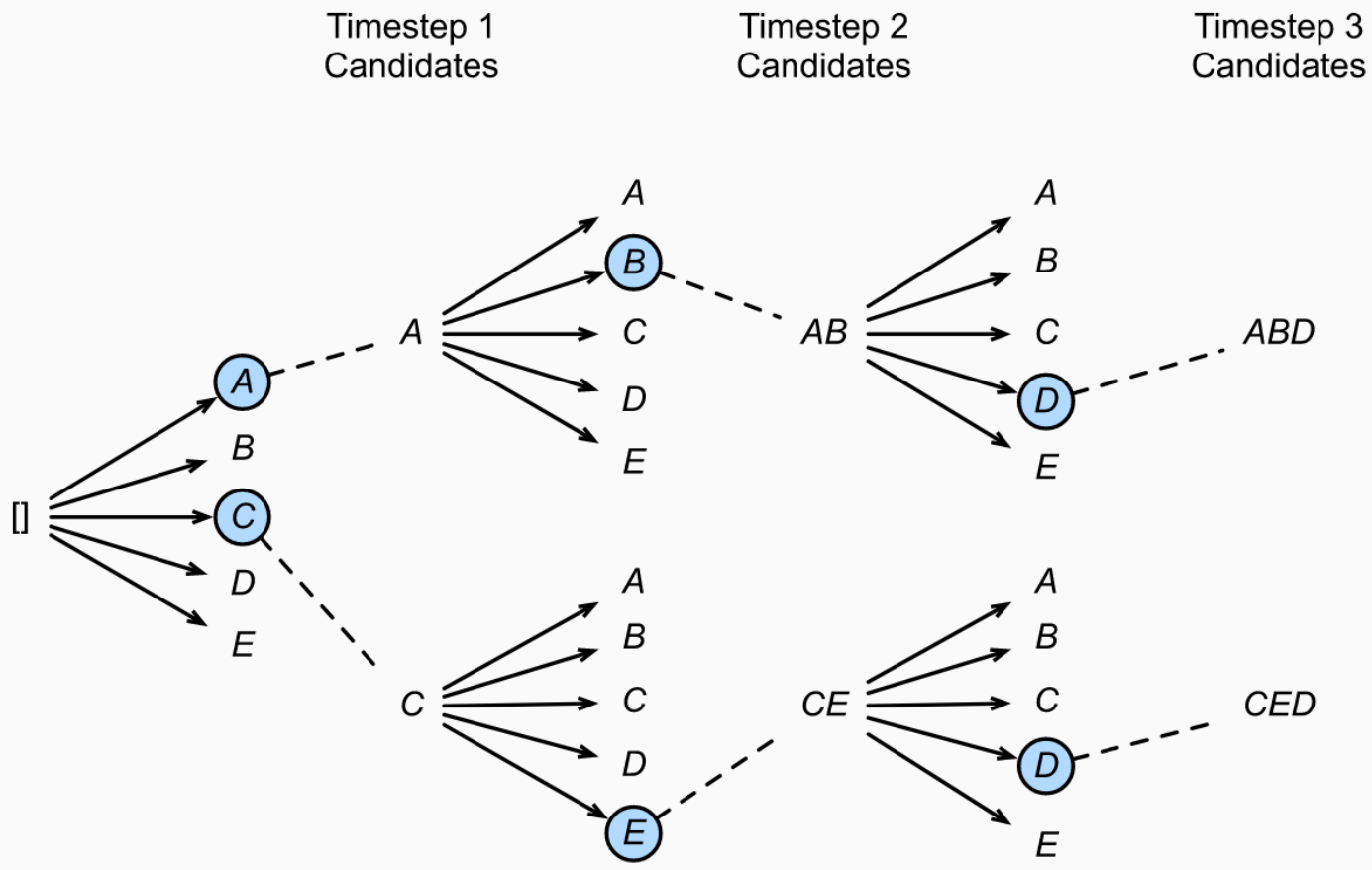
Beam Search (cont.)

k=2



Beam Search (cont.)

k=2



Beam Search (cont.)

- The computational overhead of beam search is between greedy search and exhaustive search.
- Greedy search can be treated as a beam search with a beam size of 1.
- Beam search strikes a balance between computational overhead and search quality using a flexible beam size of k.